

KNBS Maintenance, Public Launch, and Accuracy Monitoring Plan for StatsChat-KE

Audience: KNBS technical leads, KNBS product/stakeholder leads, and ONS stakeholders

Status: Technical transition note

Date: May 2026

Purpose

This note sets out how KNBS can maintain and develop StatsChat-KE after ONS exits the project, while preserving the accuracy discipline established during the April 2026 testing work. It should be read alongside the [April 2026 accuracy report](#), the [April 2026 architecture change record](#), and the [accuracy workflow documentation](#).

The central recommendation is simple: KNBS should treat StatsChat-KE as an operated statistical service, not only as a codebase. The project now has a measurable quality baseline. That baseline should become the control point for future maintenance, model changes, corpus updates, and any decision about public launch.

Current Baseline

StatsChat-KE is no longer only a proof-of-concept with anecdotal examples. As of April 2026, it has an audited benchmark workbook, an evaluator that runs against the live API, retrieval and grounding metrics, timestamped run archives, and a cross-run ledger for comparing results over time.

The current reference configuration is the cloud API using `openai/gpt-5.4-mini`, with the April 2026 retrieval, routing, prompt, and guardrail architecture. On the expanded audited benchmark, that configuration achieved answerable accuracy of $57/61 = 0.934$, unanswerable accuracy of $13/13 = 1.000$, overall accuracy of $70/74 = 0.946$, and pipeline document hit@8 of $56/61 = 0.918$.

These numbers should not be treated as a one-off historical result. They should be treated as the approved benchmark baseline until KNBS deliberately replaces it with a newer audited baseline.

Ownership Model

For KNBS to maintain the tool safely, three responsibilities need to be explicit. The same person may hold more than one role in the short term, but the responsibilities should not be left implicit.

The product owner decides who the tool is for, whether it is internal-only, pilot, or public-facing, and what scope boundaries apply. This role owns release go/no-go decisions and prioritises improvements.

The technical maintainer owns deployment, API uptime, secrets, environment configuration, model/provider configuration in [main.toml](#), corpus update jobs, incident response, benchmark execution, and artifact retention.

The statistical or domain reviewer owns benchmark quality. This includes reviewing source text, auditing new benchmark rows, classifying real user failures, and deciding whether a wrong answer is a model failure, retrieval failure, benchmark issue, or source-data issue.

Without these responsibilities, a public launch would be operationally weak even if the software continues to run.

Supported Configuration

KNBS should freeze one supported production configuration before any wider rollout. The recommended baseline is cloud API mode, `openai/gpt-5.4-mini`, the April 2026 audited retrieval and guardrail architecture, and the current rebuilt FAISS corpus aligned with the audited benchmark and update pipeline.

Ad hoc model swaps should not be allowed in production. Provider and model behaviour changes over time, and the May 2026 OpenRouter instability around `mistralai/mistral-small-3.1-24b-instruct` showed that a route which once worked can later become unreliable. If KNBS wants to evaluate a different model, that should happen in staging and be benchmarked before release.

Release Discipline

KNBS should treat changes as release-impacting when they can change answers, retrieval, safety behaviour, or the API contract. This includes changing the cloud model, provider, prompts, retrieval logic, reranking logic, index/corpus, guardrails, or response schema.

For every release-impacting change, the safe workflow is to apply the change in staging, run the audited benchmark, compare against the approved baseline, review answer quality and retrieval metrics, and only then decide whether to promote. Manual spot checks are useful for demonstrations, but they are not a substitute for benchmark comparison.

The release gate should be conservative. KNBS should not accept a drop in unanswerable accuracy without explicit sign-off, should investigate any material drop in answerable accuracy or retrieval metrics, and should treat an unexplained increase in missing answers as a release risk. The exact numeric tolerances can be set by KNBS, but the principle should be firm: a release should not go live simply because it looks acceptable on a few examples.

Accuracy Monitoring

Accuracy monitoring should have two parts: offline benchmark monitoring and live usage monitoring. Offline benchmarking catches regressions before release. Live monitoring catches new failure modes that the benchmark does not yet cover.

The controlled benchmark asset is [StatsChat QA Verified Audited.xlsx](#). KNBS should not edit it casually in place. If benchmark rows are added or corrected, that should be recorded as a benchmark revision so that future changes in accuracy can be interpreted correctly.

The benchmark runner is [evaluate_accuracy.py](#). The standard cloud benchmark command is:

```
python tests/accuracy/evaluate_accuracy.py \
  --excel tests/accuracy/StatsChat_QA_Verified_Audited.xlsx \
  --host http://127.0.0.1:8001 \
  --api-mode cloud \
  --content-type all \
  --retrieval-k 8 \
  --timeout 420
```

The benchmark should run before every release, after every material corpus update, after every model or prompt change, and on a regular cadence even without changes. Monthly is a reasonable minimum cadence for a maintained pilot.

KNBS should retain the timestamped run folders under tests/accuracy/runs/cloud/<timestamp>/ and the cross-run ledger at tests/accuracy/runs/run_history.csv. The top-level files such as tests/accuracy/accuracy_results.csv and tests/accuracy/accuracy_results_summary.csv are useful latest-run outputs, but the timestamped archives are what make regression analysis possible.

The main metrics to track are answerable accuracy, unanswerable accuracy, overall accuracy, pipeline doc hit@8, first and any reference document match, first and any reference page hit, answer coverage, and answer missing rate. These are already emitted by the evaluator and preserved in run summaries.

Live Monitoring

Once the tool is used by real users, benchmark monitoring will not be enough. KNBS should also monitor live query volume, latency, failure rate, refusal rate, user feedback, and sampled answer quality.

The API already exposes useful runtime state through /health, including the configured API mode and model. This matters because KNBS needs to confirm that a live system is actually running the approved configuration. The server deployment docs and API code also support persistent logging through STATSCHAT_LOG_FILE and STATSCHAT_LOG_DIR; those should be enabled for any serious pilot or public deployment.

Live review should include regular sampling of real questions. A technical maintainer and a domain reviewer should classify sampled outputs as correct answers, incorrect answers, correct refusals, cases that should have refused, unclear sources, or source-correct answers with poor wording. Repeated failures should feed back into benchmark revisions and product decisions.

Corpus and Benchmark Governance

StatsChat quality depends heavily on the indexed KNBS corpus. KNBS should operate the ingestion and update pipeline with discipline: update PDFs from approved KNBS sources, process them to JSON, rebuild or incrementally update the FAISS store, and run benchmark checks after material updates. Operational references include the [Operating Manual](#) and [server deployment documentation](#).

If a public-facing service is planned, KNBS should maintain separate staging and production indexes. Staging should be used to validate corpus updates before promotion to production.

The audited benchmark should also evolve, but under control. KNBS should add new audited rows when users repeatedly ask important question types not represented in the current benchmark, when new publication families are added, or when a major failure mode is discovered. At the same time, KNBS should avoid benchmark inflation. A stable baseline benchmark is needed for regression checks, and new rows should be added in controlled revisions with clear notes.

Scanned PDFs and OCR Risk

One important limitation is that many statistical PDFs are not born-digital text documents. Some are scanned images, or contain scanned tables embedded inside otherwise searchable PDFs. In those cases the current pipeline may fail to extract text and tables properly. If the answer is trapped in an image, the retrieval system may never see it, and the language model cannot reliably answer from it.

This is not primarily an LLM problem. It is a source-data and ingestion problem. A stronger model cannot recover information that was never extracted into the JSON corpus or vector index.

KNBS should therefore treat PDF text quality as part of the production data pipeline. Where possible, KNBS should publish born-digital, text-searchable PDFs generated directly from the source document rather than scanned copies. For legacy scanned reports, KNBS should add an OCR stage before indexing, using a tool that can preserve page numbers and extract tables with enough structure to support citation and verification.

OCR should not be treated as automatically reliable. The staging pipeline should flag pages with low extracted-text volume, suspicious character noise, or missing table content. Those reports should be reviewed before promotion to production, especially if they cover high-value statistics. When OCR is used, KNBS should keep the OCR output and metadata so that errors can be traced and corrected later.

This also affects benchmark governance. If a benchmark question depends on a scanned table, KNBS should confirm that the answer text is actually present in the indexed JSON. If it is not, the row should either be excluded from retrieval-based accuracy measurement until OCR is fixed, or explicitly labelled as an ingestion/OCR failure rather than an LLM answer failure.

Public Launch Position

The current system should not move directly from prototype status to unrestricted public use. A safer rollout path is internal KNBS use, guided stakeholder demonstrations, a limited pilot with selected external users, and only then public launch if operational monitoring and release governance are stable.

Before public launch, KNBS should be able to answer yes to the following questions: is there a named product owner, technical maintainer, and domain reviewer; is there a supported production model configuration; is the benchmark run before release; is live usage logged and reviewed; is there an incident path for wrong answers or degraded performance; and is there a rollback path if a model or provider change degrades quality?

If several of those answers are no, a limited pilot is safer than public release.

Operating Cadence

The minimum operating cadence does not need to be large, but it must be consistent. A practical starting point is weekly or fortnightly corpus update review, monthly audited benchmark runs, monthly live query quality review, and a benchmark run before every release.

The most practical short-term plan is for KNBS to freeze the approved gpt-5.4-mini cloud configuration, nominate the owner roles, keep the audited benchmark as the release control asset, retain run artifacts, enable API logging, and use internal or guided access before considering unrestricted public access.

Bottom Line

KNBS can maintain and develop StatsChat-KE further, and it can potentially launch the tool more broadly. That should only happen if the system is treated as an operated service with measurable quality controls.

The most important asset ONS leaves behind is not only the code. It is the April 2026 measured baseline, the audited benchmark, and the evaluation workflow that allow KNBS to detect when the system improves, drifts, or regresses.